

Social Sandbox: Full-Stack Architecture & Routing Blueprint

This document outlines your secure API flow, the complete suite of training modules, the ML libraries you'll use, and the absolute fastest way to get your Vultr GPU running.

The Secure Architecture Flow

To minimize time spent messing with Linux servers, we are treating Vultr as a "Dumb AI Engine."

1. **The Frontend (Vue/Vanilla JS):** Runs in the user's browser. It sends requests to your Flask Backend (`http://localhost:5000` during dev).
2. **The Backend (Flask):** Runs locally on your laptop (or deployed on Render). It holds your API keys and system prompts. Flask makes an HTTP request to your Vultr server (`http://<VULTR_IP>:11434/api/generate`).
3. **The Engine (Vultr + Ollama):** Does the heavy lifting, processes the LLaMA 3 prompt, and returns text to Flask. Flask formats it and sends it back to the frontend.

The Complete Training Module Suite

To impress the judges, group your modules into three distinct "Gyms". This makes your MVP look like a massive, feature-rich platform.

1. The Digital Comm Suite (Texting)

Uses the reusable Vue.js Chat component.

- **Sarcasm Sweeper:** User reads a pre-generated chat and uses checkboxes to identify passive-aggressive or sarcastic texts.
- **The Icebreaker:** User is dropped into a chat with a stranger (LLaMA 3) and must successfully introduce themselves and find a common interest in 5 texts.
- **Hint Hunter:** The AI starts dropping subtle hints that it needs to leave (e.g., "Wow, look at the time", "Anyway..."). User must recognize it and end the conversation politely.
- **The Ping-Pong Protocol:** User must keep a conversation going for 6 turns without asking a "closed-ended" (yes/no) question.

2. The Vocal Sandbox (Audio & Speech)

Uses the Walkie-Talkie UI with ElevenLabs TTS.

- **Tone Check (Conflict Resolution):** User must verbally de-escalate an angry coworker or boss.

- **The Pitch (Confidence Training):** User must introduce themselves verbally. The system grades them on filler words ("um", "uh") and vocal pauses (stuttering).

3. The Facial Gym (Computer Vision)

Uses the split-screen Webcam UI.

- **Expression Reader (Static):** A fast-paced quiz where users identify complex emotions from static images.
- **Face Emotions Helper (Live Masking):** User is given a scenario ("Your friend got a promotion"). They must hold the socially appropriate expression ("Happy") for 3 seconds on camera.

ML Models & Frontend Integrations (No API Keys Needed)

For the client-side features (Facial tracking and Speech confidence), you don't need complex backend ML. Use these open-source browser tools:

1. Facial Emotion Recognition (The "Face Emotions Helper")

- **Tool:** `face-api.js`
- **Link:** github.com/justadudewhohacks/face-api.js
- **How to use:** Download the pre-trained weights (`tiny_face_detector` and `face_expression_model`) from their repo and put them in your `/models` folder. The library runs instantly in the browser via a `<video>` tag and returns an array of emotions (e.g., `{ happy: 0.98, sad: 0.01 }`).

2. Speech Confidence & Stuttering Tracker

- **The "Hackathon" Approach:** Do not try to build an audio-waveform analyzer. Use a combination of two browser APIs:
 - **Web Speech API (Native):** Set `recognition.interimResults = true` .When the user says "um" or "uh", the API transcribes it. Send this raw, unedited transcript to LLaMA 3 with the prompt: *"Count the filler words and assess the confidence of this transcript."*
 - **Hark.js (Audio Events):** github.com/latentflip/hark. This is a tiny library that detects when a user *starts* and *stops* speaking. Use it to time the gaps between their words. If there are 5+ gaps longer than 1.5 seconds, send a "High Stutter/Hesitation" flag to your Flask backend!

The Flask API Route Map (`app.py`)

Keep Flask lean. It just formats data and acts as a traffic cop.

- `POST /api/users/xp` -> Increments Mongo database XP.
- `POST /api/analyze_chat` -> Receives chat history and `module_type`. Flask attaches the correct LLaMA 3 prompt and sends it to Vultr. Returns JSON feedback.
- `POST /api/analyze_speech` -> Receives text transcript and "pause data" from Hark.js. Sends to Vultr to grade confidence.
- `POST /api/generate_audio` -> Sends text to ElevenLabs API, returns an MP3 URL.

5-Minute Vultr Speedrun (Minimal Effort)

Since you want to spend as little time as possible on the server, we are just going to install Ollama, open the port, and leave it alone.

1. Deploy Server: Deploy a **Cloud GPU (NVIDIA A16)** with **Ubuntu 22.04 LTS**. **2. SSH In:** `ssh root@<YOUR_VULTR_IP>` **3. Install Ollama:** `curl -fsSL https://ollama.com/install.sh | sh` **4. Download LLaMA 3:** `ollama run llama3` (type `/bye` when it finishes).

5. The Magic Step (Expose Ollama to your Flask App): By default, Ollama blocks outside traffic. To let your locally-running Flask app talk to it, run these three commands on your Vultr server:

1. `mkdir -p /etc/systemd/system/ollama.service.d`
2. `echo -e "[Service]\nEnvironment=\"OLLAMA_HOST=0.0.0.0\" " > /etc/systemd/system/ollama.service.d/override.conf`
3. `systemctl daemon-reload && systemctl restart ollama`

You are done with Vultr. You can close the terminal.

Now, in your Python Flask code (running on your laptop), you just make HTTP requests to:
`http://<YOUR_VULTR_IP>:11434/api/generate`